ASP.Net Core App. Hosting

Overview



Hosting Models

-In Process

-Out of Process

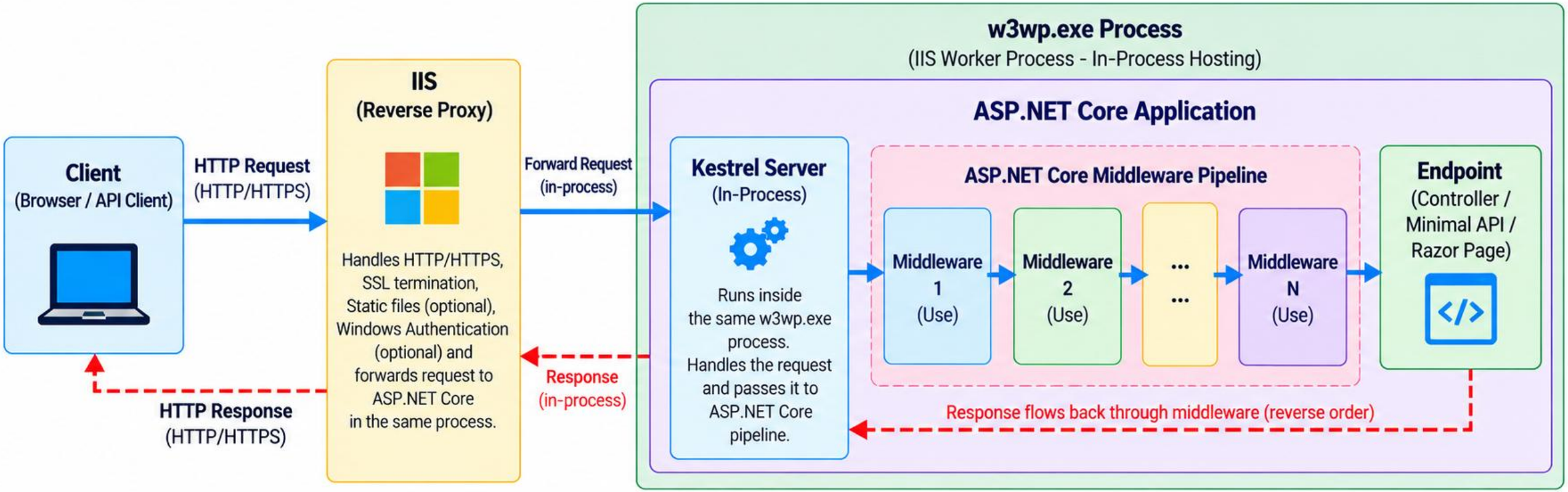
How to Configure Hosting?

Hosting Models

1. In Process:

- CreateBuilder() method calls **UseIIS()** method and host the app inside of the IIS worker process (**w3wp.exe or iisexpress.exe**).
- From a performance standpoint, InProcess hosting delivers significantly higher request throughput than out of Process hosting.
- In the case of IIS, the process name that executes the app is w3wp and in the case of IIS Express it is iisexpress
- To get the process name executing the app, use System.Diagnostics.Process.GetCurrentProcess().ProcessName
- When we are run the project from Visual Studio it uses IISExpress by default.
- IIS Express is a lightweight, self-contained version of IIS, optimized for application development. We do not use it for production. In production we use IIS.

In-Process Hosting Model in ASP.NET Core

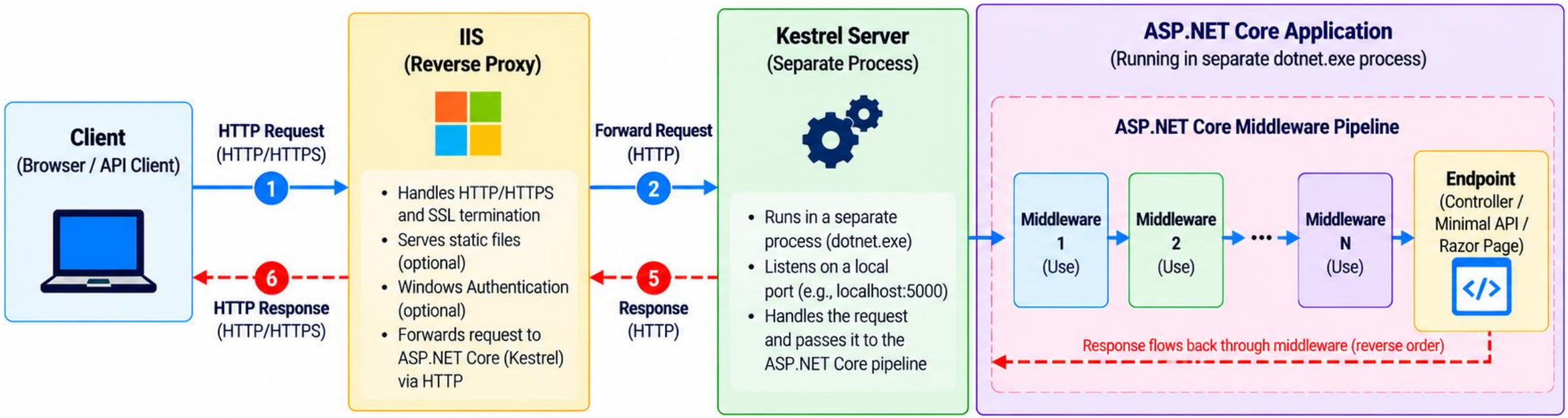


- In-Process hosting runs the ASP.NET Core application inside the IIS worker process (w3wp.exe).
- IIS forwards the request to Kestrel inside the same process.
- Request goes through the ASP.NET Core middleware pipeline, reaches the endpoint, and the response flows back in reverse order.
- No separate Kestrel process is created (unlike Out-of-Process hosting).

2. Out-Of- Process:

- The out-of-process hosting model in ASP.NET Core runs your application in a separate worker process isolated from the web server handling incoming requests. In this configuration, an external web server (like IIS, Nginx, or Apache) acts as a reverse proxy, while the internal Kestrel server processes the application logic

Out-Of-Process Hosting Model in ASP.NET Core



1 Client Request

Client sends an HTTP/HTTPS request to the application URL (e.g., https://example.com).

2 IIS (Reverse Proxy)

IIS receives the request, handles SSL termination and other configurations, and forwards the request to Kestrel using HTTP (e.g., http://localhost:5000).

3 Kestrel (Separate Process)

Kestrel runs in a separate dotnet.exe process, listens on a local port and passes the request to the ASP.NET Core middleware pipeline.

4 Middleware Pipeline (Use)

The request goes through the configured middleware components (Use) in order and reaches the endpoint (controller / minimal API / Razor page).

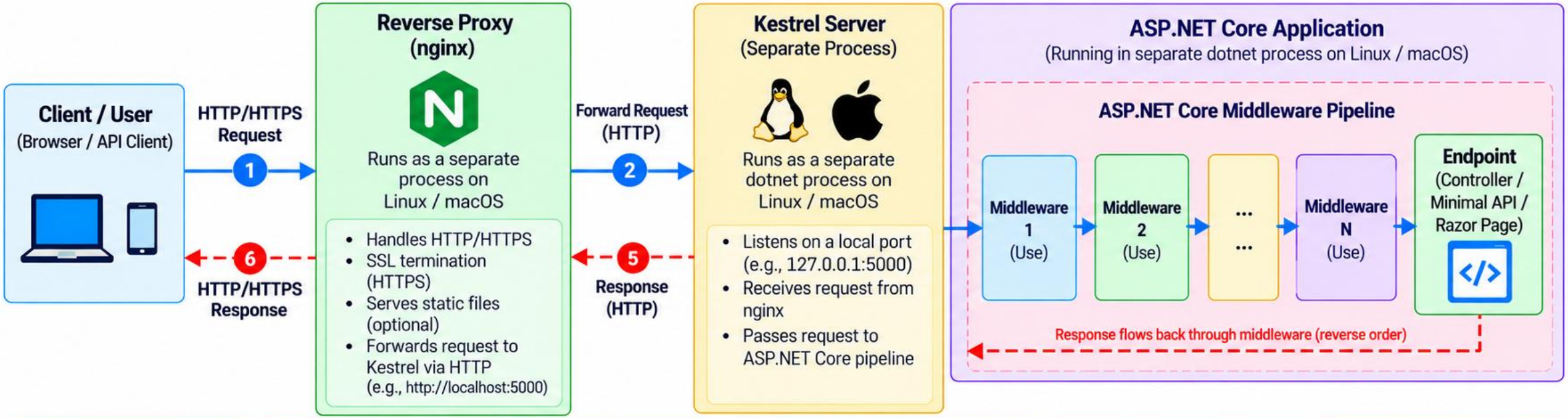
5 Response (Back to IIS)

The response is generated by the application and flows back through the middleware in reverse order to Kestrel, which sends it back to IIS.

6 Response to Client

IIS receives the response and sends it back to the client over HTTP/HTTPS.

Out-Of-Process Hosting Model in ASP.NET Core (Linux / macOS)



1 Client Request

Client sends an HTTP/HTTPS request to the application URL (e.g., https://example.com) on Linux / macOS.

2 Nginx (Reverse Proxy)

Nginx receives the request, handles SSL termination and other configurations, and forwards the request to Kestrel using HTTP (e.g., http://localhost:5000).

3 Kestrel (Separate Process)

Kestrel runs in a separate dotnet process on Linux / macOS, listens on a local port (e.g., 127.0.0.1:5000) and passes the request to the ASP.NET Core pipeline.

4 Middleware Pipeline (Use)

The request goes through the configured middleware components (Use) in order and reaches the endpoint (controller / minimal API / Razor page).

5 Response (Back to Nginx)

The response is generated by the application and flows back through the middleware in reverse order to Kestrel, which sends it back to nginx over HTTP.

6 Response to Client

Nginx receives the response and sends it back to the client over HTTP/HTTPS on Linux / macOS.

Note: On **Linux / macOS** there is **no IIS**, so a reverse proxy such as **Nginx, Apache or Caddy** is used, which forwards the request to **Kestrel** running in a separate dotnet process.

- **What is Kestrel:**

Kestrel is a cross-platform web server for ASP.NET Core. It is supported on all platforms and versions that .NET Core supports. It is included by default as internal server in ASP.NET Core. Kestrel can be used, by itself as an edge server i.e Internet-facing web server that can directly process the incoming HTTP requests from the client. In Kestrel, the process used to host the app is **dotnet.exe.**

How to Configure Hosting?

- **Project congiguration file**

Right Click to Project→Edit Project Config

.csproj

```
<Project Sdk="Microsoft.NET.Sdk.Web">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <AspNetCoreHostingModel>InProcess</AspNetCoreHostingModel>
  </PropertyGroup>
</Project>
```

References

https://learn.microsoft.com/en-us/aspnet/core/host-and-deploy/iis/in-process-hosting

https://www.c-sharpcorner.com/article/hosting-models-in-asp-net-core-3-1-applications2/

https://dotnettutorials.net/lesson/asp-net-core-inprocess-hosting/

https://dotnettutorials.net/lesson/asp-net-core-outofprocess-hosting/

https://referbruv.com/blog/exploring-aspnet-core-fundamentals-hosting-a-web-application/

https://www.learnhowtoprogram.com/c-and-net/basic-web-applications/the-previous-hosting-model-versus-the-new-minimal-hosting-model

https://code-maze.com/aspnetcore-inprocess-outofprocess-hosting-models/